TMM4128:
Machine Learning for Engineers

Andrei Lobov
e-mail: andrei.lobov@ntnu.no
phone: +47 45 913 455
room: 213, Verkstedteknisk
web: http://www.lobov.biz/academia

Slides:> http://lobov.biz/academia/mle/250221

Outline : Introduction to Artificial Neural Networks (code samples)

- Definitions / Motivation
- From Biological to Artificial Neurons
  - Biological Neurons
  - Logical Computations with Neurons
  - The Perceptron
  - Multi-Layer Perceptron (MLP) and Backpropagation
  - Regression MLPs
  - Classification MLPs
- Implementing MLPs with Keras
  - Installing TensorFlow 2
  - Building an Image Classifier Using the Sequential API
- Examples

_

Source: Aurelien Geron, “Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow : concepts, tools, and techniques to build intelligent systems”, 2019 (Available online) / **Chapter 10**

ANN : Artificial Neural Networks

ANN : nature-inspired approach for building intelligent machines.

ANNs are versatile, powerful, and scalable, making them ideal to tackle large and highly complex Machine Learning tasks, such as classifying billions of images (e.g., Google Images), powering speech recognition services (e.g., Apple's Siri), recommending the best videos to watch to hundreds of millions of users every day (e.g., YouTube), or learning to beat the world champion at the game of Go by playing millions of games against itself (DeepMind's Alpha- Zero).

List of animals by number of neurons

From Biological to Artificial Neurons

ANNs first introduced in 1943 by the neurophysiologist Warren McCulloch and the mathematician Walter Pitts: "A Logical Calculus of Ideas Immanent in Nervous Activity".

1960s: widespread belief that we would soon be conversing with truly intelligent machines.

1980s: new architectures were invented and better training techniques were developed.

1990s: other powerful Machine Learning techniques were invented, such as Support Vector Machines.

Now: yet another wave… (?)

ANN now (1)

- **Huge quantity of data available** to train neural networks, and ANNs frequently outperform other ML techniques on very large and complex problems.
- Tremendous **increase in computing power** since the 1990s,so training large neural networks in a reasonable amount of time. (Moore's Law + the gaming industry, producing / stimulating powerful GPU cards by the millions).
- **Improved training algorithms**. To be fair they are only slightly different from the ones used in the 1990s, but these relatively small tweaks have a huge positive impact.

ANN now (2)

- Some theoretical limitations of ANNs have turned out to be benign in practice. For example, many people thought that ANN training algorithms were doomed because they were likely to get stuck in local optima, but it turns out that this is rather rare in practice (or when it is the case, they are usually fairly close to the global optimum).
- ANNs seem to have entered a **virtuous circle of funding and progress.** Amazing products based on ANNs regularly make the headline news, which pulls more and more attention and funding toward them, resulting in more and more progress, and even more amazing products.

Biological Neurons (1)

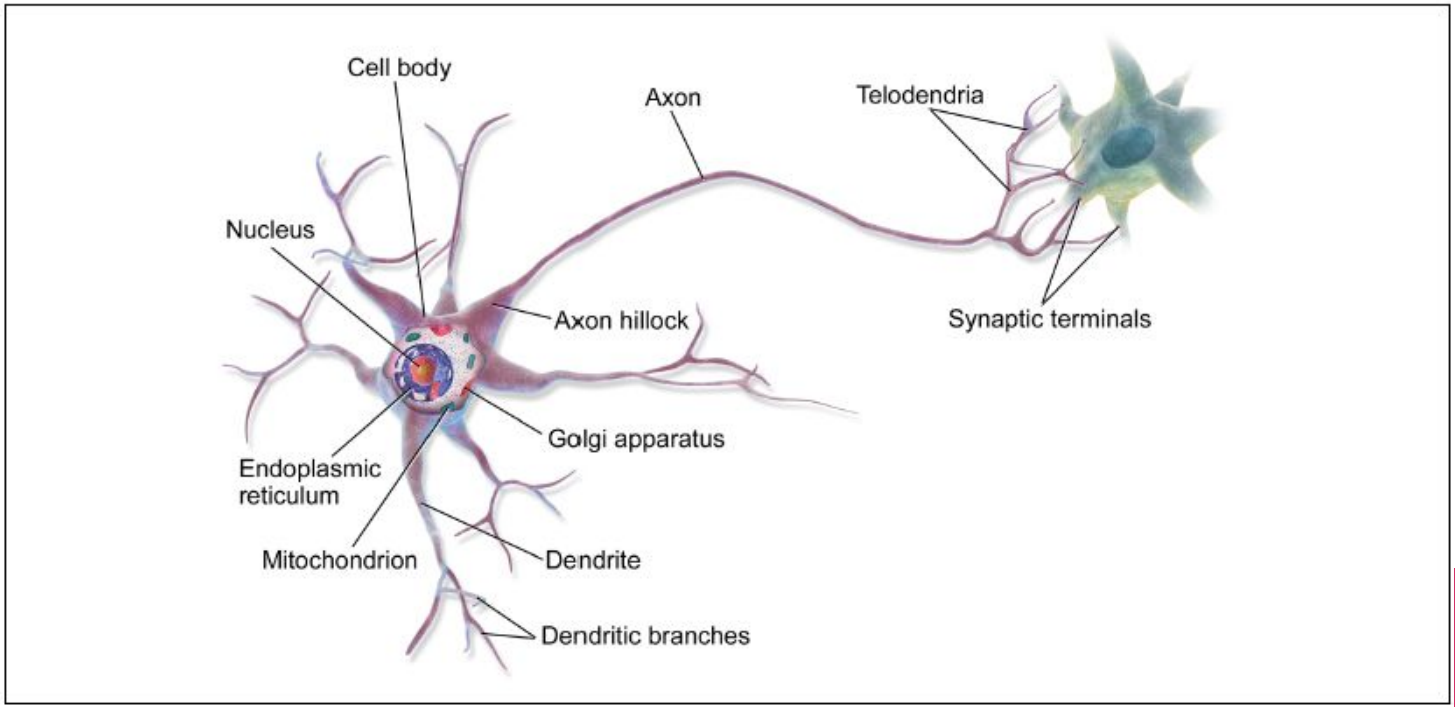


Figure 10-1. Biological neuron[3]

Biological Neurons (2)

- Biological neurons receive short electrical impulses called signals from other neurons via the synapses. When a neuron receives a sufficient number of signals from other neurons within a few milliseconds, it fires its own signals.
- Individual biological neurons behave in a rather simple way, but organized in a vast network of billions of neurons, each neuron typically connected to thousands of other neurons.
- Complex computations can be performed by a vast network of fairly simple neurons.

BNN : Biological neural network

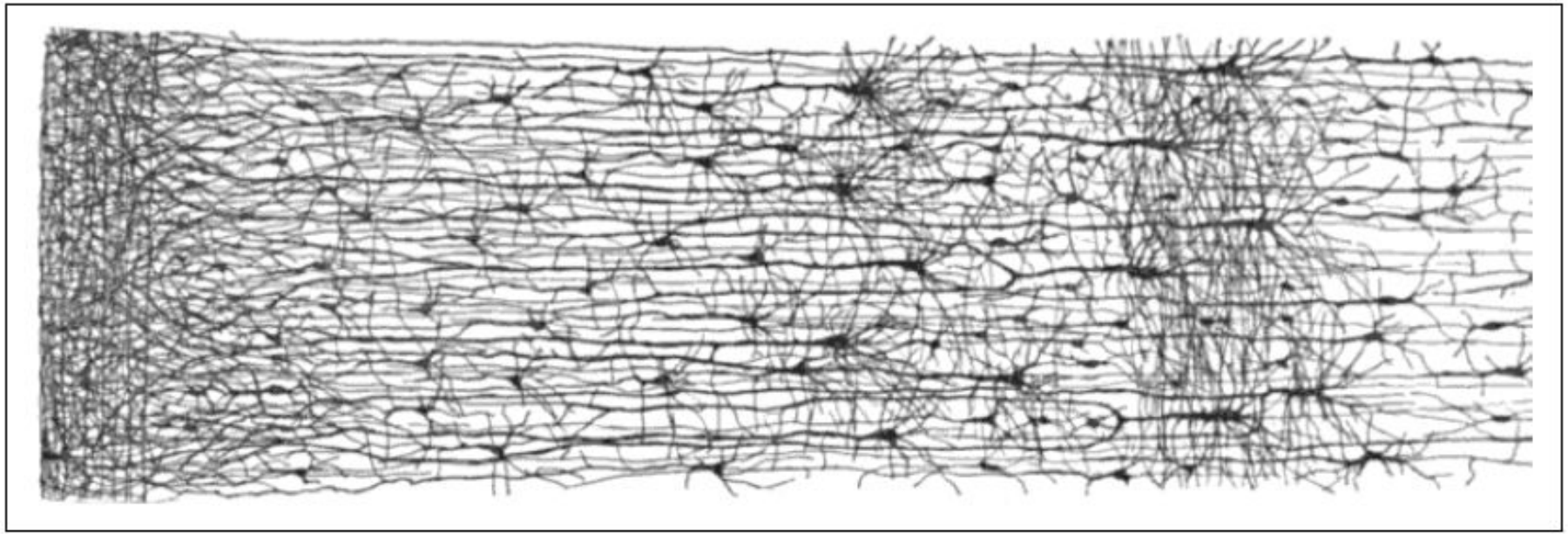


Figure 10-2. Multiple layers in a biological neural network (human cortex)[5]

Logical Computations with Neurons (1)

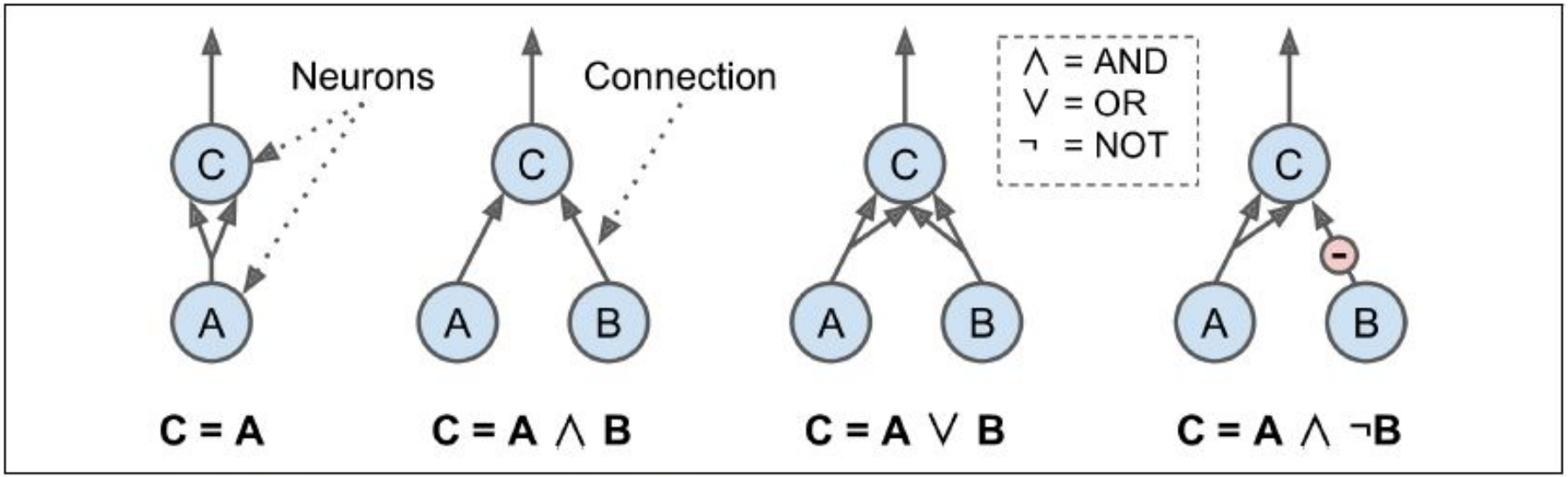


Figure 10-3. ANNs performing simple logical computations

Logical Computations with Neurons (2)

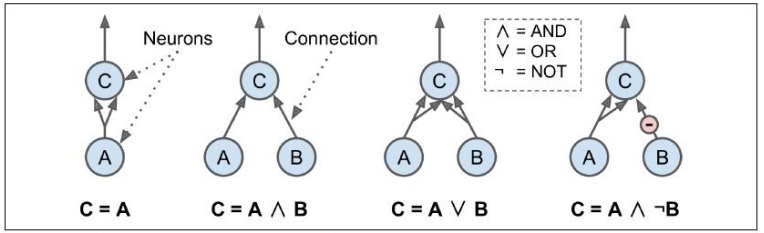


Figure 10-3. ANNs performing simple logical computations

1. If neuron A is activated, then neuron C gets activated as well (since it receives two input signals from neuron A), but if neuron A is off, then neuron C is off as well.
2. Logical AND: C is activated only when both neurons A and B are activated.
3. Logical OR: C gets activated if either neuron A or neuron B is activated (or both).
4. With inhibitor: C is activated only if neuron A is active and if neuron B is off. Logical NOT: A always on, then logical NOT for B.

The Perceptron (Steps 1-8)

- Perceptron is one of the simplest ANN architectures, invented in 1957 by Frank Rosenblat.
- Based on a **threshold logic unit (TLU)**, or sometimes a **linear threshold unit (LTU)**: the inputs and output are now numbers (instead of binary on/off values) and each input connection is associated with a weight.
- Computing a weighted sum of its inputs: $(z = w_1\, x_1 + w_2\, x_2 + \cdots + w_n\, x_n = \mathbf{x}^T\, \mathbf{w})$
- Step function to sum and output the result: $h_{\mathbf{w}}(\mathbf{x}) = \text{step}(z)$, where $z = \mathbf{x}^T\, \mathbf{w}$.

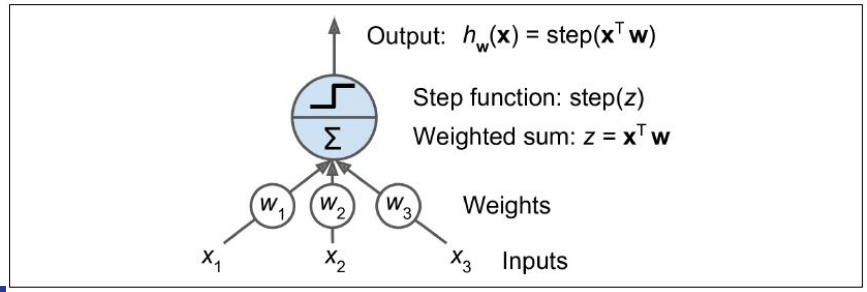


Figure 10-4. Threshold logic unit

The Perceptron : Step functions

- *Equation 10-1. Common step functions used in Perceptrons*

$$\text{heaviside}(z) = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases} \qquad \text{sgn}(z) = \begin{cases} -1 & \text{if } z < 0 \\ 0 & \text{if } z = 0 \\ +1 & \text{if } z > 0 \end{cases}$$

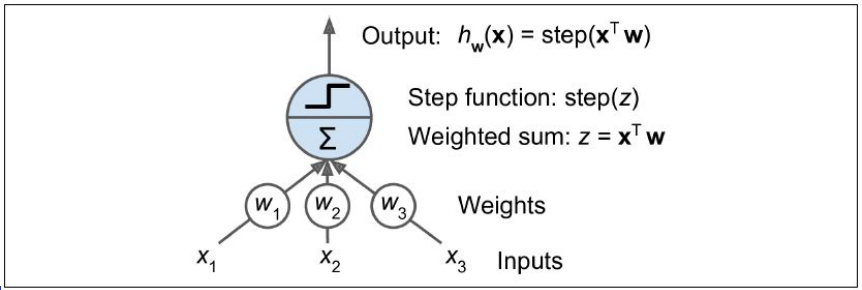


Figure 10-4. Threshold logic unit

TLU and linear binary classification

A single TLU can be used for simple linear binary classification.

1. Computes a linear combination of the inputs
2. If the result exceeds a threshold, outputs the positive class or else outputs the negative class (just like a Logistic Regression classifier or a linear SVM).

For example, one could use a single TLU to classify iris flowers based on the petal length and width (also adding an extra bias feature $x_0 = 1$). Training a TLU in this case means finding the right values for w_0, w_1, and w_2.

Perceptron diagram

A single layer of TLUs, with each TLU connected to all the inputs. When all the neurons in a layer are connected to every neuron in the previous layer (i.e., its input neurons), it is called a *fully connected layer* or a *dense layer.*

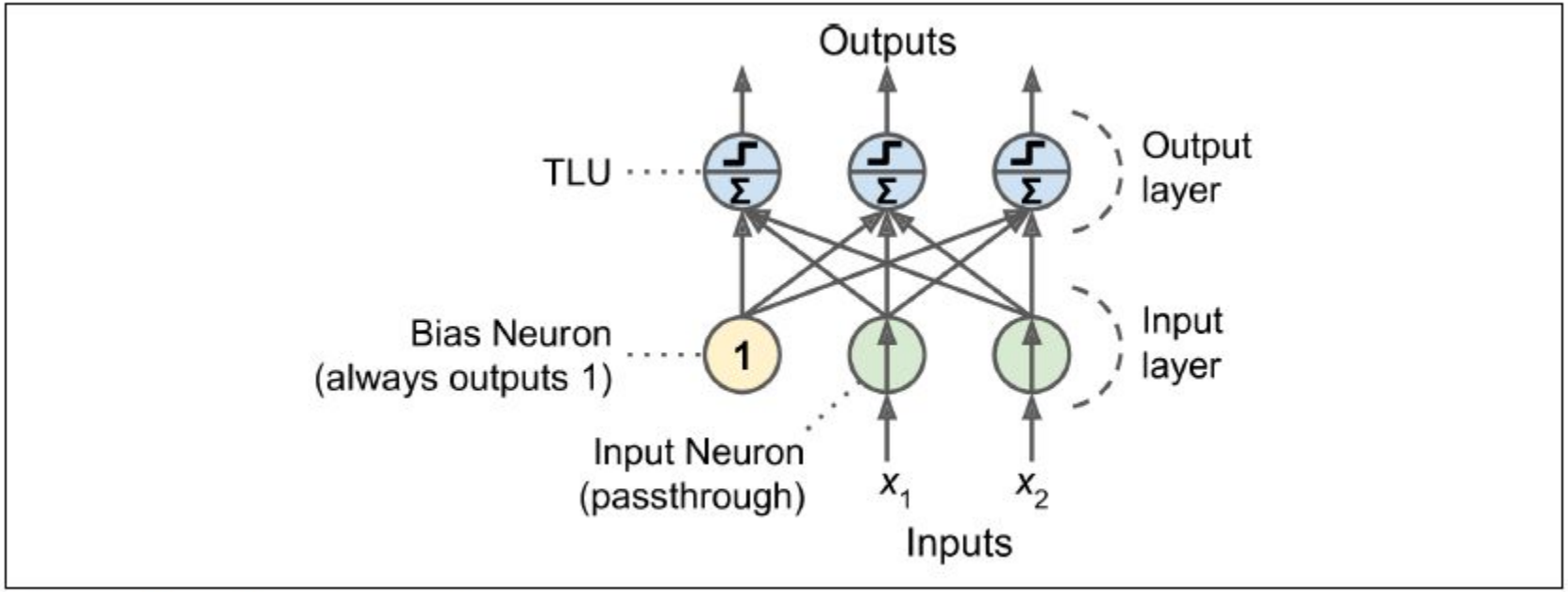


Figure 10-5. Perceptron diagram

Computing outputs

Equation 10-2. Computing the outputs of a fully connected layer

$$h_{\mathbf{W},\mathbf{b}}(\mathbf{X}) = \phi(\mathbf{XW} + \mathbf{b})$$

- **X** represents the matrix of input features. It has one row per instance, one column per feature.
- The weight matrix **W** contains all the connection weights except for the ones from the bias neuron. It has one row per input neuron and one column per artificial neuron in the layer.
- The bias vector **b** contains all the connection weights between the bias neuron and the artificial neurons. It has one bias term per artificial neuron.
- The function ϕ is called the **activation function**: when the artificial neurons are TLUs, it is a step function.

Perceptron learning rule

Equation 10-3. Perceptron learning rule (weight update)

$$w_{i,j}^{(\text{next step})} = w_{i,j} + \eta\left(y_j - \hat{y}_j\right)x_i$$

- $w_{i,j}$ is the connection weight between the i^{th} input neuron and the j^{th} output neuron.
- x_i is the i^{th} input value of the current training instance.
- is the output of the j^{th} output neuron for the current training instance.
- $\hat{y}_j$ is the target output of the j^{th} output neuron for the current training instance.
- η is the learning rate.

The decision boundary of each output neuron is linear, so Perceptrons are incapable of learning complex patterns (like Logistic Regression classifiers). If the training instances are linearly separable, converge to a solution. Perceptron Convergence Theorem.

Perceptron class

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.linear_model import Perceptron

iris = load_iris()
X = iris.data[:, (2, 3)]  # petal length, petal width
y = (iris.target == 0).astype(np.int)  # Iris Setosa?

per_clf = Perceptron()
per_clf.fit(X, y)

y_pred = per_clf.predict([[2, 0.5]])
```

Limitations and solution

1969 monograph by Marvin Minsky and Seymour Papert showed a number of serious weaknesses of Perceptrons, e.g. that they are incapable of solving some trivial problems (e.g., the Exclusive OR (XOR) classification problem) → stack multiple Perceptrons → *Multi-Layer Perceptron* (MLP)

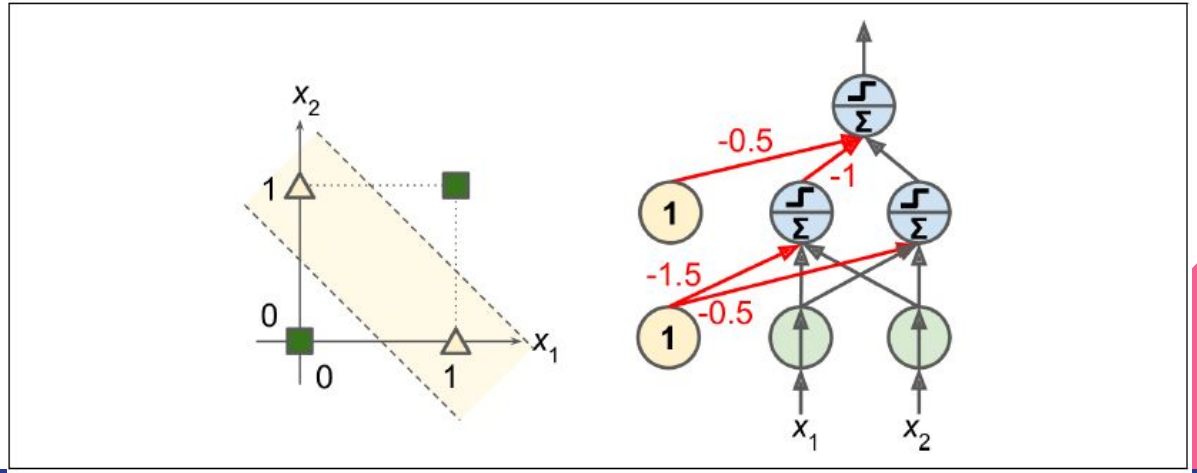


Figure 10-6. XOR classification problem and an MLP that solves it

x_1	x_2	x_1 XOR x_2
0	0	0
0	1	1
1	0	1
1	1	0

Multi-Layer Perceptron (MLP) and Backpropagation (1)

An MLP is composed of one (passthrough) *input layer*, one or more layers of TLUs, called *hidden layers*, and one final layer of TLUs called the *output layer*.

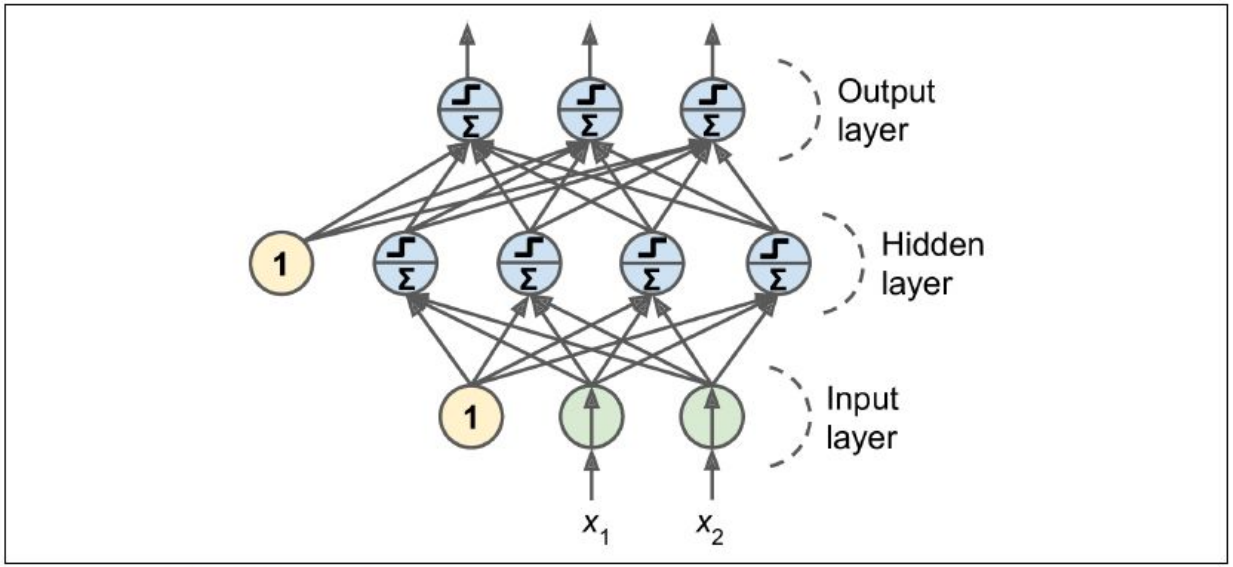


Figure 10-7. Multi-Layer Perceptron

About *Deep* Neural Network (DNN)

- An ANN with a deep stack of hidden layers, called a *deep neural network* (DNN).
- The field of Deep Learning studies DNNs, and more generally models containing deep stacks of computations. However, many people talk about Deep Learning whenever neural networks are involved (even *shallow* ones).
- Many years of struggle to find a way to train MLPs, without success.
  - Solved in 1986 with the the backpropagation training algorithm.
  - It is simply Gradient Descent using an efficient technique for computing the gradients automatically.
  - Just two passes through the network (one forward, one backward), the backpropagation algorithm is able to compute the gradient of the network's error with regards to every single model parameter. Finding out how each connection weight and each bias term should be tweaked in order to reduce the error.

Multi-Layer Perceptron (MLP) and Backpropagation (2)

- A key change to the MLP's architecture: they replaced the step function with the **logistic function**, $\sigma(z) = 1 / (1 + \exp(-z))$.
- Essential because the step function contains only flat segments, so there is no gradient to work with (Gradient Descent cannot move on a flat surface).
- So, a well-defined nonzero derivative everywhere, allowing Gradient Descent to make some progress at every step.
- Other *activation functions*:
  - The hyperbolic tangent function $\tanh(z) = 2\sigma(2z) - 1$, Output in [-1, 1]
  - The Rectified Linear Unit function: $\mathrm{ReLU}(z) = \max(0, z)$.

Multi-Layer Perceptron (MLP) and Backpropagation (3)

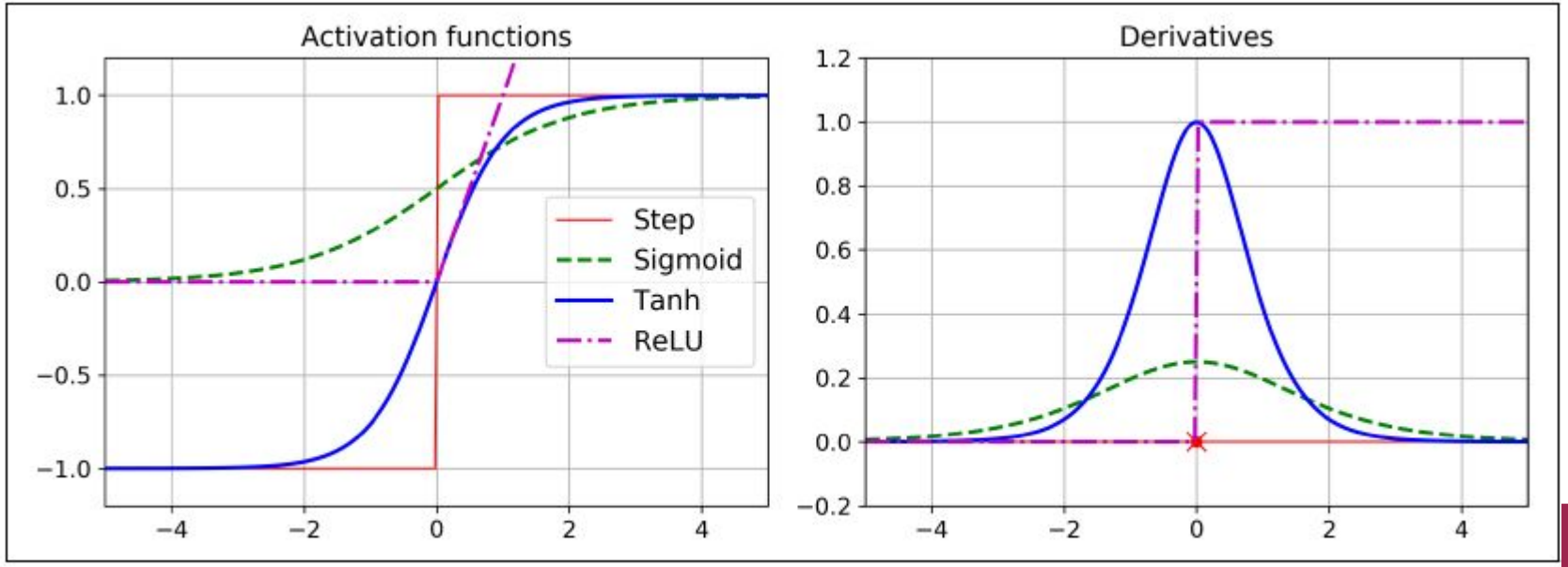


Figure 10-8. Activation functions and their derivatives

Regression MLPs (1)

- To predict a **single value** (e.g., the price of a house given many of its features), then a **single output neuron**: its output is the predicted value.
- **Multivariate regression** (i.e., to predict multiple values at once), **output neuron per output dimension**.
- Examples:
  - Locating the center of an object on a 2D image: 2 output neurons (x, y).
  - Placing a bounding box around an object: 4 output neurons (x, y, width, height).

Regression MLPs (2)

Table 10-1. Typical Regression MLP Architecture

Hyperparameter	Typical Value
# input neurons	One per input feature (e.g., 28 x 28 = 784 for MNIST)
# hidden layers	Depends on the problem. Typically 1 to 5.
# neurons per hidden layer	Depends on the problem. Typically 10 to 100.
# output neurons	1 per prediction dimension
Hidden activation	ReLU (or SELU, see Chapter 11)
Output activation	None or ReLU/Softplus (if positive outputs) or Logistic/Tanh (if bounded outputs)
Loss function	MSE or MAE/Huber (if outliers)

Classification MLPs

- **Binary classification** problem: a single output neuron using the logistic activation function: output between [0, 1], can be seen as the estimated probability of the positive class.

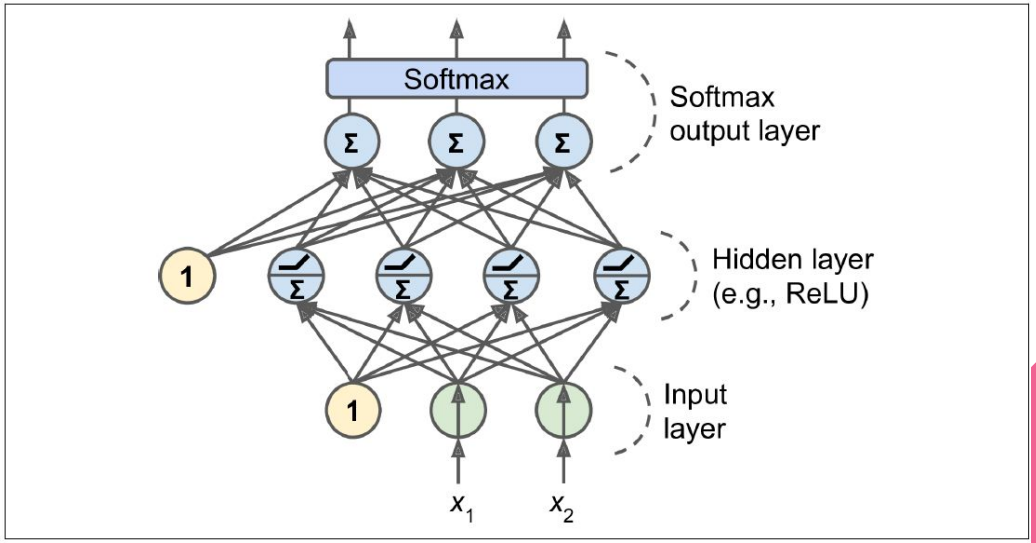


Figure 10-9. A modern MLP (including ReLU and softmax) for classification

Typical Classification MLP Architecture

Table 10-2. Typical Classification MLP Architecture

Hyperparameter	Binary classification	Multilabel binary classification	Multiclass classification
Input and hidden layers	Same as regression	Same as regression	Same as regression
# output neurons	1	1 per label	1 per class
Output layer activation	Logistic	Logistic	Softmax
Loss function	Cross-Entropy	Cross-Entropy	Cross-Entropy

On Softmax Regression classifier prediction

Equation 4-19. Softmax score for class k

$$s_k(\mathbf{x}) = \mathbf{x}^T \boldsymbol{\theta}^{(k)}$$

Equation 4-20. Softmax function

$$\hat{p}_k = \sigma(\mathbf{s}(\mathbf{x}))_k = \frac{\exp\left(s_k(\mathbf{x})\right)}{\sum_{j=1}^{K} \exp\left(s_j(\mathbf{x})\right)}$$

- K is the number of classes.
- $\mathbf{s}(\mathbf{x})$ is a vector containing the scores of each class for the instance x.
- $\sigma(\mathbf{s}(\mathbf{x}))_k$ is the estimated probability that the instance x belongs to class k given the scores of each class for that instance.

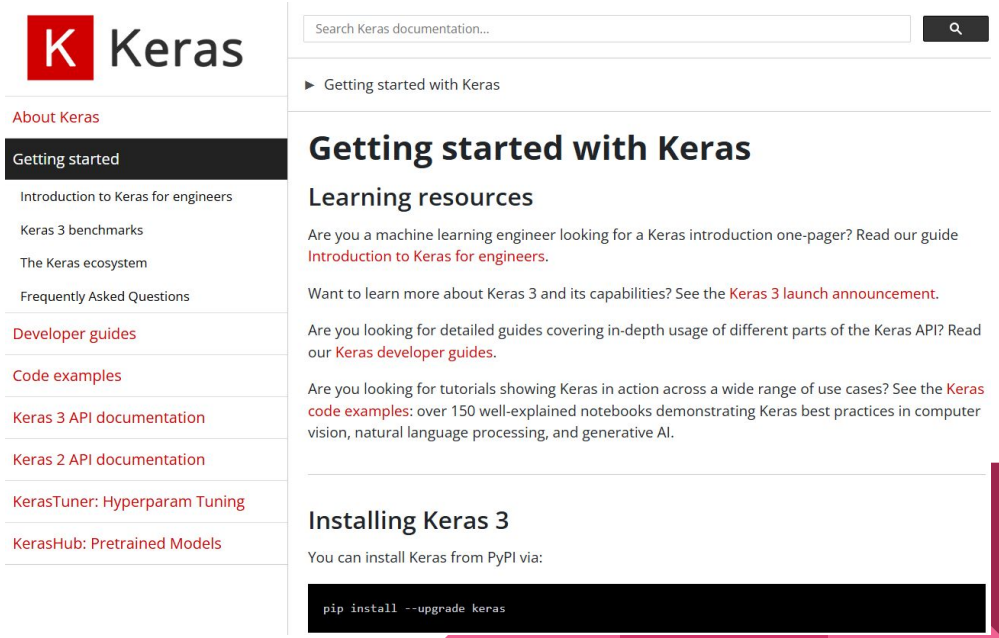
Equation 4-21. Softmax Regression classifier prediction

$$\hat{y} = \underset{k}{\operatorname{argmax}}\ \sigma(\mathbf{s}(\mathbf{x}))_k = \underset{k}{\operatorname{argmax}}\ s_k(\mathbf{x}) = \underset{k}{\operatorname{argmax}}\ \left(\left(\boldsymbol{\theta}^{(k)}\right)^T \mathbf{x}\right)$$

Implementing MLPs with Keras

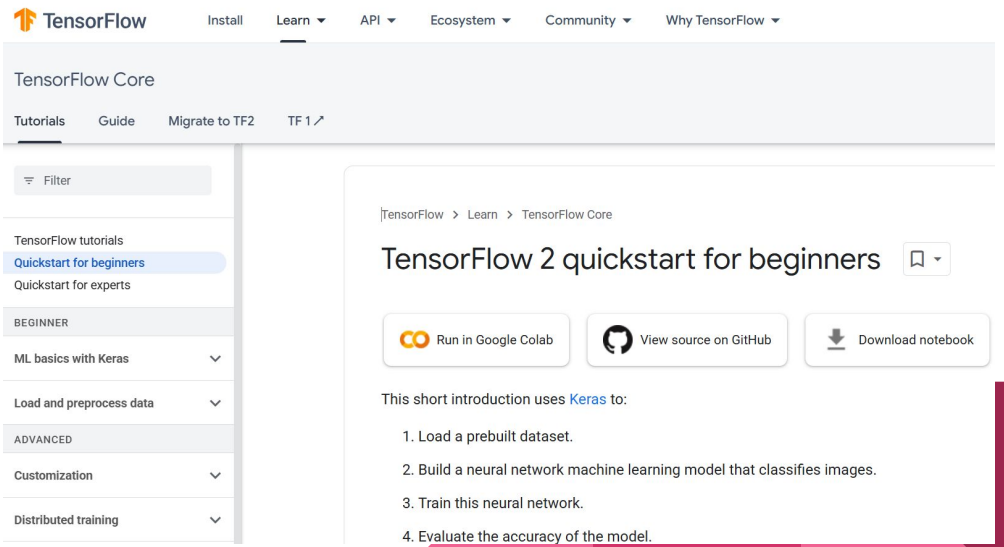
Keras

- “deep learning framework works with TensorFlow, JAX, and PyTorch interchangeably”
- https://keras.io/getting_started/



TensorFlow

- “a software library for machine learning and artificial intelligence. It can be used across a range of tasks, but is used mainly for training and inference of neural networks.”
- https://www.tensorflow.org/learn



Multibackend Keras (left) and tf.keras

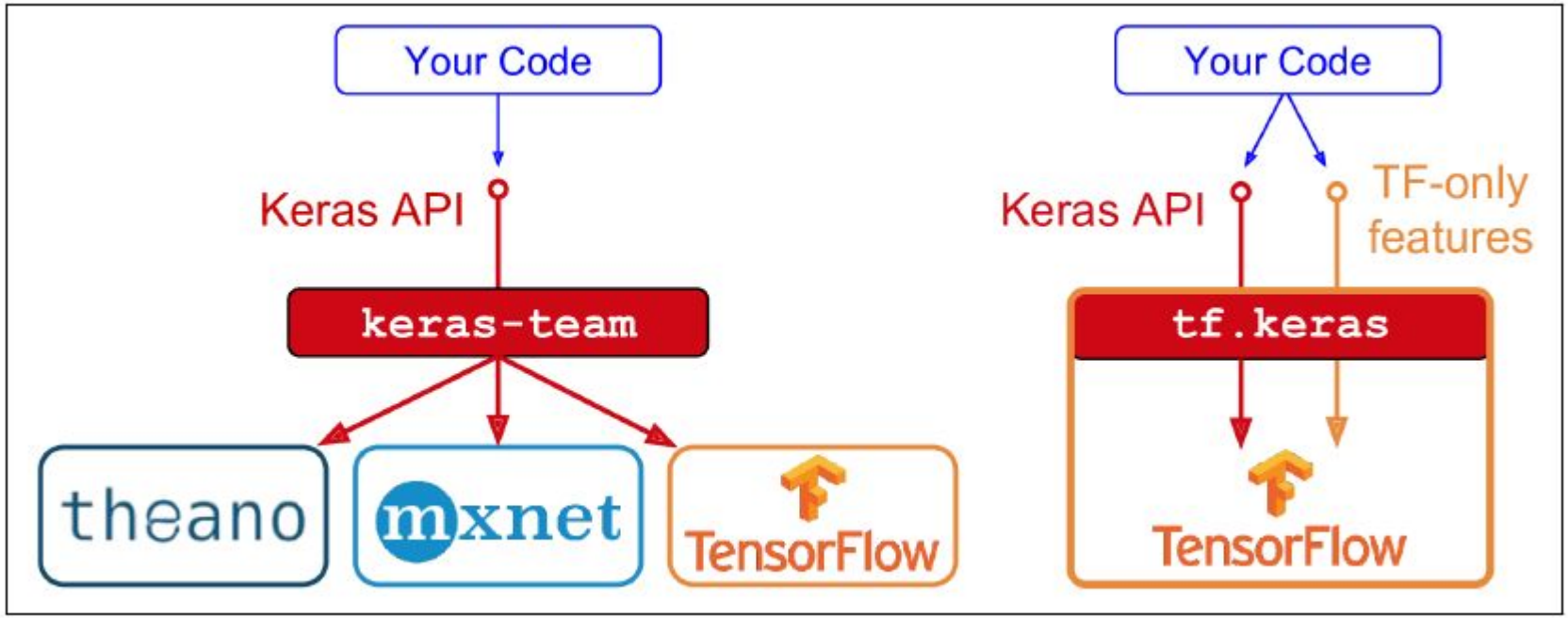


Figure 10-10. Two Keras implementations: keras-team (left) and tf.keras (right)

Installing TensorFlow

```
$ cd $ML_PATH               # Your ML working directory (e.g., $HOME/ml)
$ source env/bin/activate   # on Linux or MacOSX
$ .\env\Scripts\activate    # on Windows
```

python -m pip install --upgrade tensorflow

Testing:

```
>>> import tensorflow as tf
>>> from tensorflow import keras
>>> tf.__version__
'2.0.0'
>>> keras.__version__
'2.2.4-tf'
```

Building an Image Classifier Using the Sequential API

Fashion MNIST: the images represent fashion items rather than handwritten digits, so each class is more diverse and the problem turns out to be significantly more challenging than MNIST.

Figure 10-11. Samples from Fashion MNIST

Fashion MNIST

- Using Keras to Load the Dataset
- Creating the Model Using the Sequential API
- Compiling the Model
- Training and Evaluating the Model
- Using the Model to Make Predictions

Takeaways

- From BNN to ANN.
- Perceptron and MLP, activation functions.
- Start with Keras and Tensorflow